

AI Lab assignment 4
Tejas Patil
U24AI061

Q1)

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

vector<string> cities = {
    "Chicago", "Detroit", "Cleveland", "Indianapolis", "Columbus",
    "Pittsburgh", "Buffalo", "Syracuse", "New York",
    "Philadelphia", "Baltimore", "Boston", "Providence", "Portland"
};

int n = 14;

int adj[14][14];

struct Node {
    int state;
    Node* parent;
    int path_cost;

    Node(int s, Node* p = nullptr, int c = 0) {
        state = s;
        parent = p;
        path_cost = c;
    }
};

struct PriorityQueue {
    vector<Node*> data;

    int f(Node* node) {
```

```

        return node->path_cost;
    }

    void add(Node* node) {
        data.push_back(node);
        sort(data.begin(), data.end(),
            [&](Node* a, Node* b) {
                return f(a) < f(b);
            });
    }

    Node* pop() {
        Node* best = data.front();
        data.erase(data.begin());
        return best;
    }

    bool is_empty() {
        return data.empty();
    }
};

struct Problem {
    int INITIAL;
    int goal;

    Problem(int i, int g) {
        INITIAL = i;
        goal = g;
    }

    bool is_goal(int state) {
        return state == goal;
    }

    vector<int> ACTIONS(int state) {
        vector<int> actions;
        for (int i = 0; i < n; i++) {
            if (adj[state][i] != 0)

```

```

        actions.push_back(i);
    }
    return actions;
}

int RESULT(int state, int action) {
    return action;
}

int ACTION_COST(int s, int s_prime) {
    return adj[s][s_prime];
}
};

vector<Node*> EXPAND(Problem& problem, Node* node) {
    vector<Node*> children;

    for (int action : problem.ACTIONS(node->state)) {
        int cost = node->path_cost + problem.ACTION_COST(node->state,
action);
        Node* child = new Node(action, node, cost);
        children.push_back(child);
    }
    return children;
}

pair<Node*, int> BEST_FIRST_SEARCH(Problem& problem) {

    Node* node = new Node(problem.INITIAL);

    PriorityQueue frontier;
    frontier.add(node);

    unordered_map<int, Node*> reached;
    reached[problem.INITIAL] = node;

    int explored = 0;

```

```

while (!frontier.is_empty()) {
    node = frontier.pop();
    explored++;

    if (problem.is_goal(node->state)) {
        return {node, explored};
    }

    for (Node* child : EXPAND(problem, node)) {
        int s = child->state;
        if (!reached.count(s) || child->path_cost <
reached[s]->path_cost) {
            reached[s] = child;
            frontier.add(child);
        }
    }
}

return {nullptr, explored};
}

```

```

vector<string> get_path(Node* node) {
    vector<string> path;
    while (node) {
        path.push_back(cities[node->state]);
        node = node->parent;
    }
    reverse(path.begin(), path.end());
    return path;
}

```

```

int main() {

    memset(adj, 0, sizeof(adj));

    vector<tuple<int,int,int>> edges = {
        {0,1,283},{0,2,345},{0,3,182},
        {1,2,169},{1,6,256},

```

```

        {2,6,189},{2,5,134},{2,4,144},
        {3,4,176},
        {4,5,185},
        {5,6,215},{5,9,305},{5,10,247},
        {6,7,150},
        {7,8,254},{7,11,312},
        {8,9,97},{8,12,181},
        {9,10,101},{9,11,215},
        {11,12,50},{11,13,107}
    };

    for (auto [u,v,w] : edges) {
        adj[u][v] = w;
        adj[v][u] = w;
    }
    int start = 7; // Syracuse
    int goal  = 0; // Chicago
    Problem problem(start, goal);
    auto [solution, explored] = BEST_FIRST_SEARCH(problem);
    cout << "Best First Search Path:\n";
    vector<string> path = get_path(solution);
    for (auto& c : path)
        cout << c << " -> ";
    cout << "END\n";

    cout << "Total cost: " << solution->path_cost << "\n";
    cout << "Number of paths explored: " << explored << "\n";

    return 0;
}

```

Best First Search Path:

Syracuse -> Buffalo -> Cleveland -> Chicago -> END

Total cost: 684

Number of paths explored: 15

Q2)

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int x, y;
    int h;
};

class prio_queue {
    vector<Node> heap;

public:
    bool empty() {
        return heap.empty();
    }

    void push(Node n) {
        heap.push_back(n);
        int i = heap.size() - 1;

        while (i > 0) {
            int parent = (i - 1) / 2;
            if (heap[parent].h <= heap[i].h)
                break;
            swap(heap[parent], heap[i]);
            i = parent;
        }
    }

    Node pop() {
        Node root = heap[0];
        heap[0] = heap.back();
        heap.pop_back();

        int i = 0;
        int size = heap.size();

        while (true) {
            int left = 2 * i + 1;
```

```

        int right = 2 * i + 2;
        int smallest = i;

        if (left < size && heap[left].h < heap[smallest].h)
            smallest = left;
        if (right < size && heap[right].h < heap[smallest].h)
            smallest = right;

        if (smallest == i)
            break;

        swap(heap[i], heap[smallest]);
        i = smallest;
    }
    return root;
};

int heuristic(int x, int y, int gx, int gy) {
    return abs(x - gx) + abs(y - gy);
}

int main() {

    vector<vector<int>> grid = {
        {1,1,1,1,1,1,1,1,1},
        {1,0,0,1,0,0,0,0,1},
        {1,1,0,1,1,0,1,1,1},
        {1,0,0,0,0,0,0,0,1},
        {1,1,0,1,1,0,1,0,1},
        {1,1,0,1,0,0,0,0,1},
        {1,1,1,1,1,1,1,1,1}
    };

    int rows = grid.size();
    int cols = grid[0].size();
    int sx = 5, sy = 2;    // Entry
    int gx = 3, gy = 7;    // Exit

    prio_queue pq;

```

```

vector<vector<bool>> checked(rows, vector<bool>(cols, false));
vector<vector<pair<int,int>>> parent(rows,vector<pair<int,int>>(cols,
{-1,-1}));

pq.push({sx, sy, heuristic(sx, sy, gx, gy)});
checked[sx][sy] = true;

int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

while (!pq.empty()) {
    Node curr = pq.pop();

    if (curr.x == gx && curr.y == gy) {
        cout << "Exit reached\n";
        break;
    }

    for (int i = 0; i < 4; i++) {
        int nx = curr.x + dx[i];
        int ny = curr.y + dy[i];

        if (nx >= 0 && ny >= 0 && nx < rows && ny < cols) {
            if (!checked[nx][ny] && grid[nx][ny] == 0) {
                checked[nx][ny] = true;
                parent[nx][ny] = {curr.x, curr.y};
                pq.push({nx, ny, heuristic(nx, ny, gx, gy)});
            }
        }
    }
}

vector<pair<int,int>> path;
int x = gx, y = gy;

while (!(x == sx && y == sy)) {
    path.push_back({x, y});
    auto p = parent[x][y];
    x = p.first;
    y = p.second;
}

```

```
}  
path.push_back({sx, sy});  
reverse(path.begin(), path.end());  
  
cout << "Evacuation Path:\n";  
for (auto &p : path)  
    cout << "(" << p.first << "," << p.second << ") ";  
  
return 0;  
}
```

Exit reached

Evacuation Path:

(5,2) (4,2) (3,2) (3,3) (3,4) (3,5) (3,6) (3,7)